



Department of Physics, IIT Indore

Weekly Report

AUTHOR

Prince Kumar (230051013)

SUPERVISOR

Dr. Shashank Gupta

2026-05-23

Abstract

This weekly report documents the progress made in Week 2 of the quantum key distribution (QKD) network simulation project. The primary focus was mastering Discrete Event Simulation (DES) via the Sequence framework and exploring essential network components. Key milestones included implementing event-driven architectures covering Timeline management, event scheduling, and process handling and analyzing fundamental network building blocks such as Quantum/Classical Channels, object-oriented Protocols, and configurable Memory components. These foundational concepts establish the necessary architecture for the subsequent implementation of the QKD network simulation and the BBM92 protocol.

Contents

Table of Contents	ii
1 Introduction	1
2 Activities and Findings	1
2.1 Discrete Event Simulation Overview	1
2.1.1 Core Components	2
2.2 Practical Implementation: Store Class	2
2.2.1 Basic Store Implementation	2
2.2.2 Event Scheduling	3
2.2.3 Automated Event Scheduling	4
2.2.4 Timeline Termination	5
2.2.5 Logging and Debugging	5
2.3 Network Components and Protocols	7
2.3.1 Network Components Overview	7
2.3.2 Object-Oriented Protocol Architecture	8
2.3.3 Message Enumeration and Communication	9
2.3.4 Practical Implementation: Ping-Pong Protocol	10
2.3.5 Quantum Hardware Example	11
3 Key Findings & Application to QKD	12
3.1 Discrete Event Simulation	12
3.2 Network Components and Protocols	13
3.3 Application to QKD Simulation	14
4 Implementation Artifacts	14
5 Next Steps and Future Work	15
6 Conclusion	16

1 Introduction

This report documents the second week of the quantum key distribution (QKD) network simulation project. As outlined in the previous week's recommendations, the focus of this week was two-fold: (1) gaining a comprehensive understanding of Discrete Event Simulation (DES) systems through the official Sequence framework tutorial, and (2) exploring the core network components and protocols available in the Sequence framework for quantum network simulation.

DES is a fundamental paradigm for simulating systems where events occur at discrete points in time, and the state of the system changes only in response to these events. This understanding is critical for implementing the QKD network simulation, which will rely heavily on event-driven architecture.

Additionally, understanding the network components (channels, protocols, memory systems, and detectors) and the object-oriented architecture used to compose these components is essential for implementing the quantum network topology. The study of message enumeration patterns and protocol communication provides the tools needed to build reliable and maintainable quantum protocols.

Together, these two areas provide the foundational knowledge required to progress toward implementing a functional QKD network simulation.

2 Activities and Findings

2.1 Discrete Event Simulation Overview

The Discrete Event Simulation tutorial from the Sequence framework provided a practical introduction to event-driven programming. The tutorial demonstrated how to build a simple but realistic simulation system using the core components of the Sequence kernel.

2.1.1 Core Components

The DES implementation in Sequence relies on several key components:

- **Timeline:** The Timeline class manages the simulation time and maintains a queue of scheduled events. It acts as the central coordinator for the entire simulation.
- **Event:** An Event represents an occurrence at a specific point in simulation time. Each event contains a timestamp and a process to be executed.
- **Process:** A Process encapsulates a function call with its associated arguments. Processes are executed when their corresponding events are triggered.

2.2 Practical Implementation: Store Class

The tutorial provided a practical example through the implementation of a Store class that simulates a retail store opening and closing operations. The implementation evolved through several stages of increasing complexity and functionality.

2.2.1 Basic Store Implementation

The initial implementation established the foundation:

```
from sequence.kernel.timeline import Timeline

class Store(object):
    def __init__(self, tl: Timeline):
        self.opening = False
        self.timeline = tl
```

```
def open(self) -> None:
    self.opening = True

def close(self) -> None:
    self.opening = False
```

The basic Store class maintains an `opening` state and provides methods to transition between open and closed states. The Timeline object provides access to simulation time and scheduling capabilities.

2.2.2 Event Scheduling

The tutorial demonstrated how to schedule events on the Timeline:

```
from sequence.kernel.event import Event
from sequence.kernel.process import Process

tl = Timeline()
store = Store(tl)

# Create and schedule an opening event at time 7
open_proc = Process(store, 'open', [])
open_event = Event(7, open_proc)
tl.schedule(open_event)

# Run the simulation
```

```
tl.run()

print(tl.now(), store.opening)  # Output: 7 True
```

This demonstrates the basic workflow: create a Timeline, instantiate objects, create processes that reference methods on those objects, wrap processes in events with specific timestamps, schedule events, and run the simulation.

2.2.3 Automated Event Scheduling

The tutorial progressed to a more sophisticated implementation where events schedule subsequent events automatically:

```
class Store:

    def __init__(self, tl: Timeline):
        self.opening = False
        self.timeline = tl

    def open(self) -> None:
        self.opening = True
        # Schedule closing 12 time units from now
        process = Process(self, 'close', [])
        event = Event(self.timeline.now() + 12, process)
        self.timeline.schedule(event)

    def close(self) -> None:
        self.opening = False
        # Schedule opening 12 time units from now
```

```
process = Process(self, 'open', [])  
event = Event(self.timeline.now() + 12, process)  
self.timeline.schedule(event)
```

This version demonstrates event-driven cascading, where each event triggers the scheduling of the next event, creating a continuous cycle of state transitions.

2.2.4 Timeline Termination

The tutorial also covered simulation control mechanisms using Timeline termination parameters:

```
tl = Timeline(60) # Simulation ends at time 60  
tl.show_progress = False # Disable progress bar  
store = Store(tl)  
process = Process(store, 'open', [])  
event = Event(7, process)  
tl.schedule(event)  
tl.run()
```

2.2.5 Logging and Debugging

The final version integrated the Sequence logging framework for better visibility into simulation execution:

```
import sequence.utils.log as log  
  
class Store:  
    def __init__(self, tl: Timeline):
```



```
self.opening = False
self.timeline = tl

def open(self) -> None:
    if self.timeline.now() >= 60:
        self.timeline.stop()

    log.logger.info('Store being opened.')
    if self.opening == True:
        log.logger.warning('Store was already open.')

    self.opening = True
    process = Process(self, 'close', [])
    event = Event(self.timeline.now() + 12, process)
    self.timeline.schedule(event)

def close(self) -> None:
    if self.timeline.now() >= 60:
        self.timeline.stop()

    log.logger.info('Store being closed.')
    if self.opening == False:
        log.logger.warning('Store was already closed.')
```

```
self.opening = False
process = Process(self, 'open', [])
event = Event(self.timeline.now() + 12, process)
self.timeline.schedule(event)
```

2.3 Network Components and Protocols

Building upon the DES foundation, this week also involved studying and implementing network components within the Sequence framework. This included exploring the core building blocks necessary for simulating networked quantum systems: channels, protocols, memory systems, and message-based communication patterns.

2.3.1 Network Components Overview

The Sequence framework provides several fundamental components for network simulation:

- **Memory:** Quantum memory components store quantum states with configurable parameters including fidelity, frequency, efficiency, coherence time, and wavelength.
- **Detector:** Detector components measure quantum states and trigger callbacks when measurements occur.
- **Channels:** Communication channels come in two types:
 - **Quantum Channels:** Transmit quantum states (photons) between nodes with configurable attenuation and distance.
 - **Classical Channels:** Transmit classical messages between nodes for protocol coordination.

2.3.2 Object-Oriented Protocol Architecture

The framework employs an object-oriented architecture where protocols inherit from a base Protocol class and are attached to Node objects:

```
from sequence.protocol import Protocol
from sequence.topology.node import Node

class PingProtocol(Protocol):
    def __init__(self, owner: Node, name: str, other_name: str,
               other_node: str):
        super().__init__(owner, name)
        owner.protocols.append(self)
        self.other_name = other_name
        self.other_node = other_node

    def init(self):
        pass

    def start(self):
        # Protocol initialization and startup logic
        pass

    def received_message(self, src: str, message: Message):
        # Handle incoming messages
        pass
```

Each protocol handles its own initialization, startup, and message reception, enabling clean separation of concerns and modular protocol design.

2.3.3 Message Enumeration and Communication

Protocol communication is facilitated through a message-passing system with typed message enumerations:

```
from enum import Enum, auto
from sequence.message import Message

class MsgType(Enum):
    PING = auto()
    PONG = auto()

# Sending a message
new_msg = Message(MsgType.PING, self.other_name)
self.owner.send_message(destination_node, new_msg)

# Receiving and handling a message
def received_message(self, src: str, message: Message):
    if message.msg_type == MsgType.PING:
        # Handle ping message
        pass
```

Message types are defined as enumerations, providing type safety and clarity in protocol implementations. This pattern prevents message type confusion and makes protocol specifications more

explicit and maintainable.

2.3.4 Practical Implementation: Ping-Pong Protocol

A practical ping-pong protocol was implemented to demonstrate protocol communication between two nodes over classical channels:

```
# Initialize timeline and nodes
```

```
t1 = Timeline(1e12)
```

```
node1 = Node("node1", t1)
```

```
node2 = Node("node2", t1)
```

```
# Create classical channels for bidirectional communication
```

```
cc0 = ClassicalChannel("cc0", t1, 1e3)
```

```
cc1 = ClassicalChannel("cc1", t1, 1e3)
```

```
cc0.set_ends(node1, node2.name)
```

```
cc1.set_ends(node2, node1.name)
```

```
# Attach protocols to nodes
```

```
pingp = PingProtocol(node1, "pingp", "pong", "node2")
```

```
pong = PongProtocol(node2, "pong", "ping", "node1")
```

```
# Schedule initial event
```

```
process = Process(pingp, "start", [])
```

```
event = Event(0, process)
```

```
t1.schedule(event)
```

```
tl.init()
```

```
tl.run()
```

2.3.5 Quantum Hardware Example

The framework also enables implementation of quantum-specific components. An example implementation demonstrates quantum state transmission via photons:

```
class SenderNode(Node):  
    def __init__(self, name, timeline):  
        super().__init__(name, timeline)  
        memory_name = name + ".memory"  
        memory = Memory(memory_name, timeline, fidelity=1,  
                        frequency=0,  
                        efficiency=1, coherence_time=1, wavelength  
                        =500)  
        self.add_component(memory)  
        memory.add_receiver(self)  
        self.sender = Sender(self, memory_name)  
  
class ReceiverNode(Node):  
    def __init__(self, name, timeline):  
        super().__init__(name, timeline)  
        detector_name = name + ".detector"  
        detector = Detector(detector_name, timeline, efficiency=1)
```

```
self.add_component(detector)

self.set_first_component(detector_name)
```

3 Key Findings & Application to QKD

The week's work provided several important insights spanning both DES fundamentals and network simulation, and highlights how these apply to the QKD project:

3.1 Discrete Event Simulation

- **Timeline Management:** The Timeline class provides a robust mechanism for managing simulation time and event scheduling. It automatically processes events in chronological order, eliminating the need for manual time management.
- **Event Encapsulation:** By encapsulating state changes in discrete events, complex systems can be modeled as sequences of state transitions. This abstraction simplifies reasoning about system behavior.
- **Cascading Events:** Events can schedule subsequent events, enabling the creation of complex behavioral patterns without explicit control flow. This pattern is fundamental to discrete event simulation.
- **Simulation Time vs. Wall-Clock Time:** Simulation time (as tracked by the Timeline) is independent of wall-clock time, allowing simulations to run at arbitrary speeds.
- **Logging and Observability:** The Sequence framework provides built-in logging capabilities that are essential for understanding and debugging simulations. Logs are timestamped with simulation time, providing a clear record of events in chronological order.

- **State Validation:** The Store example demonstrated the importance of state validation through warnings when state transitions occur unexpectedly (e.g., attempting to open an already-open store).

3.2 Network Components and Protocols

- **Component Architecture:** The Sequence framework provides pre-built components (Memory, Detector, Channels) that can be configured with specific physical parameters, enabling realistic simulation of quantum hardware properties.
- **Protocol Inheritance Model:** Protocols inherit from a base Protocol class and override key methods (init, start, received_message), providing a clean and extensible architecture for implementing various quantum protocols.
- **Type-Safe Messaging:** Message types are defined as Enum classes, providing compile-time clarity and runtime type safety for protocol communication, preventing message misinterpretation.
- **Modular Component Composition:** Complex systems are built by composing simple components (Nodes with Memories, Detectors, and Protocols), enabling flexible system design and easy modification of individual components.
- **Channel Flexibility:** The distinction between Quantum and Classical channels enables separation of quantum state transmission from classical coordination messages, mirroring real quantum network architectures.
- **Parameterized Component Behavior:** Components can be configured with physical parameters (fidelity, coherence time, wavelength, attenuation), allowing simulations to explore the effects of realistic hardware limitations.

3.3 Application to QKD Simulation

The Discrete Event Simulation concepts and network component patterns learned this week are directly applicable to the QKD network simulation:

- **Network Events:** In the QKD simulation, network events such as photon transmission, reception, and measurement will be represented as discrete events on the Timeline.
- **Quantum State Management:** Quantum states at different nodes can be managed similarly to the Store's opening/closing state, with events triggering state transitions and processes coordinating actions across components.
- **Timing Precision:** The Timeline's ability to handle events at precise simulation times is essential for accurately modeling quantum protocols with specific timing requirements.
- **Scalability:** The event-driven architecture demonstrated in the Store and component examples scales naturally to complex systems with multiple nodes and interactions.

4 Implementation Artifacts

The work completed this week has been documented in the following files:

- **DES Tutorial:**
 - **des.ipynb:** A Jupyter notebook containing the complete DES tutorial with interactive examples and explanations.
 - **des.py:** A standalone Python script implementation of the final Store class, suitable for execution outside the notebook environment. The script demonstrates proper logging configuration and simulation execution.

- **Network Components and Protocols:**

- **hardware.ipynb:** A Jupyter notebook containing exploratory implementations of network components, channels, and quantum hardware simulation examples.
- **hardware.py:** Implementation of quantum network simulation with Sender and Receiver nodes, demonstrating quantum state transmission and measurement via photon detectors.
- **pingpong.py:** Implementation of a ping-pong protocol demonstrating classical message exchange between nodes over bidirectional classical channels. This serves as a template for more complex protocol implementations.

5 Next Steps and Future Work

In the next phase of our work, we will focus on **Chapter 3: Entanglement Management**.

In previous chapters, we introduced the usage of hardware models. In this chapter, we will use protocols in the entanglement management module to control these hardware devices and change the entanglement state of quantum memories. We will also use a simple manager protocol to control the entanglement generation protocols. We will show:

- How to use `EntanglementGenerationA` (by default creates Barret-Kok generation protocol) to entangle memories on different nodes.
- How to use `BBPSSW` (BBPSSW purification protocol) to improve the fidelity of entanglement.
- How to use `EntanglementSwappingA` and `EntanglementSwappingB` (swapping protocol) to extend the distance of entanglement.

6 Conclusion

Week 2 successfully established a comprehensive understanding of Discrete Event Simulation through the Sequence framework’s DES tutorial and produced practical insights into quantum network components. The DES work (Timeline management, event scheduling, process handling, cascading events, and logging) provides a solid foundation for implementing time-sensitive protocol logic in the QKD simulator. In parallel, exploration of network components (quantum/classical channels, configurable quantum memories, detectors, and the protocol/message architecture (illustrated by the ping-pong example and the Sender/Receiver hardware sketches)) showed how physical hardware models map to simulated components and which parameters (fidelity, efficiency, coherence time, attenuation) are most important for fidelity analysis.

Together, these results leave the project well-positioned to proceed: the DES primitives give us reliable scheduling and observability, while the hardware component models supply the necessary abstractions to implement BBM92 and entanglement-management protocols. The next steps are to integrate the hardware models into a simple topology, implement a basic BBM92 exchange, and validate key metrics (key rate and fidelity) under varying hardware parameters.